3. Gyakorlat

Python kurzus

Lista, string,

tuple, dictionary, set

Műveletek listákkal

```
a = [1,2,3,4,5]
```

Operátorok: +, *

Tartalmazás-vizsgálat: `if 3 in a:` / `not in`

A lista hossza: `len(a)`

A lista i. eleme: `elem = a[i], a[i] = elem`

Indexelés: `a[0], a[1], a[-1], a[-2], …`

A listaszelet i.-től j-1.-ig, k-nként: `a[i:j:k], a[2:6], a[::2]`

Listaelemek visszafelé: `a[::-1]`

Listaelemek törlése: `del a[0], del a[:3]`

Lista törlése: `del a`
`print(a)` → `Error!`

Példák

Hivatkozás másolás (copying): `a = [1,2,3,4,5]` `b = a`

Szeletelés (slicing): `c = a[:]`

Rendezés: `d = sorted(a,reverse=True)`

Zip:

```
m = [1,2,3,4,5]
n = ['a','b','c','d','e']
print(list(zip(m, n)))
```

```
[(1,'a'),(2,'b'),(3,'c'),(4,'d'),(5,'e')]
```

List comprehension (lista generátor):

```
j=[i**2 for i in range(10) if i%3==0]
k=[i**2 if i%3==0 else 2*i for i in range(10)]
```

```
[0, 9, 36, 81]
[0, 2, 4, 9, 8, 10, 36, 14, 16, 81]
```

```
odd = [1, 3, 5]
even = [2, 4, 6]
a=zip(odd, even)
print(a)
print(type(a))
```

?

```
for x,y in zip(odd, even):
    print(x, y, end=" ")
```

1 2 3 4 5 6

```
for elem in zip(odd, even):
    print(elem, end=" ")
```

(1, 2) (3, 4) (5, 6)

List comprehension példák:

```
számok = [i for i in range(10)]

páros_számok = [i*2 for i in range(10)]
ps = [i for i in range(10) if i % 2 == 0]
páratlan_számok = [i*2+1 for i in range(10)]
ptl = [i for i in range(10) if i % 2 == 1]

négyzet = [i**2 for i in range(10)]
ps_négyzet = [i**2 for i in range(10) if i % 2 == 0]
ptl_négyzet = [i**2 for i in range(10) if i % 2 == 1]
köbszámok = [i**3 for i in range (11)]

fibo = [1, 1]
[fibo.append(fibo[-1] + fibo[-2]) for i in range(8)]

törtek = [i/10 for i in range(11)]
tört = [60/i for i in range(1,7)]
tört_eg = [60//i for i in range(1,7)]
```

számok =	**[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]**
páros_számok =	**[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]**
ps =	**[0, 2, 4, 6, 8]**
páratlan_számok =	**[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]**
ptl =	**[1, 3, 5, 7, 9]**
négyzet =	**[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]**
ps_négyzet =	**[0, 4, 16, 36, 64]**
ptl_négyzet =	**[1, 9, 25, 49, 81]**
köbszámok =	**[0, 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]**
fibo =	**[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]**
törtek =	**[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]**
tört =	**[60.0, 30.0, 20.0, 15.0, 12.0, 10.0]**
tört_eg =	**[60, 30, 20, 15, 12, 10]**

Összehasonlító operátorok, in operátor, identitás operátorok

```
l1 = [0, 1, 2, 1]
l2 = [0, 1, 2]
l3 = [0, 1, 2]
print(l1 == l2)
print(l1 != l2)
print(l2 is l1)
print(l2 in l1)
print(l2 == l3)
print(l2 is l3)
print(l1 < l2)
print(l1 > l2)
```

```
False
True
False
False
True
False
False
True
```

Műveletek listákkal

Példák: indexelés, append, del, insert, len, pop

```
a = [1, 2, [3, 4], [5]]
print(a[2])
print(a[2][0])
print(len(a))
```

```
a = [1, ['alma',5,[3,4], 6], [2]]
print(a[1][0][2])
print(a[1][2][0])
```

```
a = [0, 1, 2, 3, 4, 5]
a.pop(-1)
a.insert(0, [6, 7])
del a[1]
a.append(0)
print(a)
```

```
a = [1, 2, 3, 4, 5]
b = []
add = 0
for num in a:
    add += num
    b.append(add)
print(b)
```

Szeletelés – Slicing

```
b = ['a','b','c','d','e','f']

print(b[1:4])
print(b[4:])
print(b[:4])
print(b[3:-2])
print(b[-3:4])
print(b[::2])
print(b[1::2])

print(b[::-1])
```

Copying - Slicing

list_1 = ["A","B","C"]
list_2 = list_1
list_3 = list_2
del list_1[0]
del list_2[0]
print(list_1)
print(list_2)
print(list_3)
	list_1 = ["A","B","C"]
	list_2 = list_1
	list_3 = list_2
	del list_1[0]
	del list_2
	print(list_1)
	#print(list_2)
	print(list_3)
		list_1 = ["A","B","C"]
		list_2 = list_1
		list_3 = list_2
		del list_1[0]
		del list_2[:]
		print(list_1)
		print(list_2)
		print(list_3)
			list_1 = ["A", "B", "C"]
			list_2 = list_1[:]
			list_3 = list_2[:]
			del list_1[0]
			del list_2[1]
			print(list_1)
			print(list_2)
			print(list_3)

Függvények-metódusok: sum, sort, reverse, remove, pop, min, max, index, sorted

a, b, c = 8, 1, 3 lista = [a, c, b] print(sum(lista)) lista.sort() print(lista)	a, b, c, d = "A", "B", "C", " " lista = [a, b, c, d] lista.reverse() lista.remove("B") ### print(lista)	lista = [2,4,6,8] lista.pop(0) print(min(lista))	l = ["A", "B", "C", "D"] print(max(l)) print(l.index("E")) **# Error!** c = sorted(l, reverse=True) print(c)

https://www.w3schools.com/python/python_ref_list.asp

1. feladat: Címletekre bontás

Írj programot, amely beolvas egy pénzösszeget, majd ezt a **lehető legnagyobb** címletekre bontja. A címletek legyenek: 20, 10, 5, 2 és 1 ezer Ft, valamint 500, 200, 100, 50, 20, 10 és 5 Ft.
Írasd ki, hogy melyik címletből mennyit és milyen összegben használt fel.
Használd a **lista.pop() metódust** a címletek lista utolsó elemének eltávolításához.

```
Kérem az összeget (Ft): 125475
Címletezve:
6 db 20000 Ft = 120000 Ft
1 db 5000 Ft = 5000 Ft
2 db 200 Ft = 400 Ft
1 db 50 Ft = 50 Ft
1 db 20 Ft = 20 Ft
1 db 5 Ft = 5 Ft
```

Használj while ciklust.
Teszteld a kódot a megadott adatok alapján.

Stringműveletek

Összefűzés: `x = "Alma" + "fa" → "Almafa"`

Sokszorozás: `y = "Rin" + 2*" Tin" → "Rin Tin Tin"`

String hossza: `hossz = len(y) → 11`

az i. karakter: `kar = x[2] → "m"`

az i.-től a j-1.-ig tartó szelet: `x[2:5] → "maf"`

példa:

```
l = "Halloween"
print(l[0:5])
print(l[-4:])
```

Metódusok:

https://www.w3schools.com/python/python_ref_string.asp

Műveletek stringekkel példák

szó = 'Hello'
print(len(szó))
 	
üres = ''
print(len(üres))
 	
én = 'I\'m'
print(len(én))
	multiline = '''Sor #1
	Sor #2'''
	print(len(multiline))
	
	kar_1 = 'A'
	kar_2 = ' ' # space
	print(ord(kar_1))
	print(ord(kar_2))
	
	print(chr(97))
	print(chr(945))
		#Indexelés
		szó = 'almafa'
		for i in range(len(szó)):
		  print(szó[i],end=' ')
		print()
		
		#Elemenkénti iteráció
		szó = 'almafa'
		for kar in szó:
		  print(kar, end='*')
		print()
			# Szeletelés - Slicing
			
			betűk = 'abcdef'
			print(betűk[1:4])
			print(betűk[4:])
			print(betűk[:4])
			print(betűk[3:-2])
			print(betűk[-3:4])
			print(betűk[::2])
			print(betűk[1::2])
			
			print(betűk[::-1])

```
eg, leb = 12, 1.2
ste = str(eg)
stl = str(leb)
print(ste + stl)

ste, stl = '12', '1.2'
eg = int(ste)
leb = float(stl)
print(eg + leb)
```

'10' == '010'	'10' == 10
'10' > '010'	'10' != 10
'10' > '8'	'10' == 1
'20' < '8'	'10' != 1
'20' < '80'	'10' > 10 ###

Exception kezelés példa

```
try:
   érték = int(input("Kérek egy számot: "))
   print(1 / érték)
except ValueError:
   print(„Helytelen bemenet.")
except ZeroDivisionError:
   print(„Nagyon helytelen bemenet.")
except:
   print("Mi történt?")
```

2. feladat: Szómanipuláló

Kérj be egy szót. (Az angol ABC betűit használd a feladatban.)

1. Válogasd szét a magánhangzóit és a mássalhangzóit két csoportba, és írd ki ezeket.
2. Készíts egy tükörszót, vagyis az eredeti szóhoz ragaszd hozzá a megfordított szót. Pl. szó = „kabala” → új_szó = „kabalaalabak”
3. Rendezd ABC rendbe a szó betűit, és írd ki a kapott stringet.
4. Titkosítsd a szót úgy, hogy a betűit rendre a betűrendben utána következő betűvel helyettesíted, pl. „alma” → „bmnb”
   A „z” betű helyett az „a” következzen, pl. „azaz” → „baba”

   (Caesar-rejtjel)

A Python stringek immutable típusúak:

```
ABC = "abcdefghijklmnopqrstuvwxyz"

del ABC[0]
ABC.append("A")
ABC.insert(0, "A")
```

Nem működnek!

Minimum, maximum érték stringben:

```
print(min("aAbB...yYzZ"))
```

.

```
dal = "Tavaszi szél vizet áraszt"
print('[' + min(dal) + ']')
print('[' + max(dal) + ']')
```

[ ]

[é]

```
ABC = "abcdefghijklmnopqrstuvwxyz"

print("d" in ABC)
print("A" in ABC)
print("0" in ABC)
print("mno" in ABC)
print("x" in ABC and "y" in ABC)
print("x" in ABC or "Y" in ABC)
print(" " not in ABC)    # space
```

Stringből lista, listából string:

```
print(list("abcabc"))
```

['a', 'b', 'c', 'a', 'b', 'c']

```
print("-".join(["alma", "körte", "szilva"]))
```

alma-körte-szilva

Az index metódus:

```
print("aAbByYzZaA".index("b"))
#print("aAbByYzZaA".index("c"))
```

2

ValueError: substring not found

```
print("aAbByYzZaA".index("A"))
```

1

A count , capitalize, center, endswith, startswith és find metódusok:

print("abcabc".**count**("b"))	2
print('abcabc'.count("d"))	0
print('aBcD'.**capitalize**())	Abcd
print('[' + 'labda'.**center**(10) + ']')	[labda]
print('[' + 'labda'.center(10, "*") + ']')	[**labda***]
if "almafa".**endswith**("fa"): print("igen")	igen
if "almafa".**startswith**("al"): print("igen")	igen
print("labda".**find**("da"))	3
print("labda".find("fa"))	-1 ###
print('labda'.find('a', 2))	4

Az isalpha, isdigit, isalnum, islower, isupper és isspace metódusok:

print("Anna".**isalpha**())	True
print('Anna20'.isalpha())	False
print('2024'.**isdigit**())	True
print("08:25".isdigit())	False
print('labda10'.**isalnum**())	True
print('#'.isalnum())	False
print('labda_10'.isalnum())	False
print(''.isalnum())	False
print("Anna".**islower**())	False
print("Bagoly".**isupper**())	False
print(' \n '.**isspace**())	True
print(" ".isspace())	True
print(" la la la".isspace())	False

Az rfind, split, lower, lstrip, rstrip, strip és replace metódusok:

print("kép kép kép".**rfind**("ké"))	8
print("kép kép kép".rfind("ké", 9))	-1 ###
print("kép kép kép".rfind("ké", 3, 10))	8

print("alma körte\neper".**split**())	['alma', 'körte', 'eper']
print("AdaT=60".**lower**())	adat=60
print("[" + " alma ".**lstrip**() + "]")	[alma]
print("elte.inf.hu".lstrip("elte."))	inf.hu
print("[" + " alma ".**rstrip**() + "]")	[alma]
print("elte.inf.hu".rstrip(".hu"))	elte.inf
print("[" + " alma ".**strip**() + "]")	[alma]
print("almalé".**replace**("lé", " lekvár"))	alma lekvár
print("almalélé".replace("lé", " lekvár"))	alma lekvár lekvár

A swapcase, title, upper metódusok

```
print("Hej Dunáról fúj a szél".swapcase())
print("Hej Dunáról fúj a szél".title())
print("Hej Dunáról fúj a szél".upper())
```

```
hEJ dUNÁRÓL FÚJ A SZÉL
Hej Dunáról Fúj A Szél
HEJ DUNÁRÓL FÚJ A SZÉL
```

sorted függvény – sort metódus

```
greek_1 = ['omega', 'alpha', 'pi', 'gamma']
greek_2 = sorted(greek_1)
print(greek_1)
print(greek_2)

greek_1.sort()
print(greek_1)
print(greek_2.sort(reverse = True))
print(greek_2)
```

```
['omega', 'alpha', 'pi', 'gamma']
['alpha', 'gamma', 'omega', 'pi’]

['alpha', 'gamma', 'omega', 'pi’]
None
['pi', 'omega', 'gamma', 'alpha']
```

Tuple: immutable

```
fruit = ("alma", "banán", "szőlő", "banán")
```

Nem működnek:

```
del fruit[0]
fruit.append("körte")
fruit.insert(0, "körte")
```

Dictionary - Kulcs: érték párok

```
import json
autó = {"marka": "Opel",
        "modell": "Astra",
        "ev": 2003,
        "tomeg": 1250}
print(autó)
print(json.dumps(autó, indent=4))
```

```
{'marka': 'Opel', 'modell': 'Astra', 'ev': 2003, 'tomeg': 1250}
{
   "marka": "Opel",
   "modell": "Astra",
   "ev": 2003,
   "tomeg": 1250
}
```

Set: nem rendezett, nem megváltoztathatóak az elemei, és nem indexelt

```
a = [1,1,1,2,2]
a = set(a)
print(a) → {1,2}
```

```
b = {2,4,6}
b.add(8) → {8,2,4,6}
```

Példák, metódusok

```
Tuple = (1, 2, 3)
print(Tuple[2])

tup = 1, 2, 3
print(tup)

a, b, c = tup
print(a * b * c)
```

```
b = {2,4,6}
b.add(8)
print(b)
b.update([10,12,12])
print(b)
b.remove(6)      ###
print(b)
b.discard(8)     ###
print(b)
b.clear()
print(b)        # set()
```

```
dict_1 = {"A": (1,5), "B": 2}
dict_2= dict_1.copy()

dict_1.clear()
print(dict_1)
print(dict_2)

del dict_1
print(dict_1)   # Error
```

```
személy = {'name': 'Merlin', 'job': 'wizard'}
for kulcs, érték in személy.items():
   print(f'What is your {kulcs}?  {érték}')
```

3. feladat: Pontszámok, átlagok, jegyek

Egy csoport munkájának értékelését végezzük el a félév közben szerzett pontok nyilvántartásából.
A **marks dictionary** tartalmazza a neveket és hozzájuk rendelt pontszámokat a [0,10] zárt intervallumból. A pontszámok listája nem üres, ha valaki nem teljesítette az adott feladatot, akkor 0 pontot kapott.

marks = { "Anna": [5,7.8], "Bea": [9.2,10], "Csaba": [7.8,9], "Dóra": [10,10], "Endre": [8,7.5], "Ferenc": [0,5]}

a) Határozd meg és írd ki hallgatóként az átlagpontszámot.

b) Írd ki a csoport átlag pontszámát.

c) Írd ki hallgatóként a jegyet a következő határok szerint:

50%-tól 2, 64%-tól 3, 78%-tól 4 és 90%-tól 5.

Köszönöm a figyelmet!

Konzultáció

Csütörtök 18:00 – 20:00